THE UNIVERSITY OF MANCHESTER
DEPARTMENT OF ELECTRICAL AND ELECTRONICS

Object Recognition for Robots: A Comparative Analysis of Traditional Local Feature Method and CNN-Based Vision

(Cognitive Robotics and Computer Vision (COMP64301) 2025/26)

Submitted By:
SOHAN DAS
14282155

Submission of: Coursework

Under the Guidance of:
Dr. Aphrodite Galata
Prof Angelo Cangelosi
Dr. Philip Zhao

A report submitted in partial fulfillment of the requirements for the degree of

Master of Science in Robotics

December,2025

Declaration of Originality

I hereby confirm that no portion of the work referred to in the thesis has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Place: Manchester, UK **Sohan Das**

Date: 03/12/2025 **Student Name**

Contents

Declaration of Originality **i**

1 Introduction **1**

2 CNN for Vision **1**

3 CNN Results **2**

4 Object Recognition Using Local Features (ORB + BoVW + SVM) **4**

5 ORB Results: Accuracy and Confusion Matrix **4**

6 Deep Learning vs Traditional Computer Vision: Comparison and Discussion **5**

7 State of the Art in Computer Vision for Robotics **6**

8 Conclusion **7**

9 References **7**

10 Appendix **8**

1. Introduction

Object recognition is a fundamental capability in modern robotics systems which enables to categorize objects in their environment and perform tasks such as grasping, navigation, human-robot interaction, etc. This report shows two major algorithms in computer vision: **Deep Learning using Convolutional Neural Networks (CNN)** and **Traditional Computer Vision using Local Features and Bag-of-Visual-Words (BoVW) with ORB descriptors**.

2. CNN for Vision

- **Dataset Preparation**

  The RGB-D dataset[1] includes multiple object categories and to ensure fair evaluation and avoid data leakage, all images from a sequence remain completely in either the training set or the test set. Using PIL to verify image integrity and eliminated 165 groups of duplicate images from our dataset. Normalization also done using ImageNet statistics.

  After cleaning, we obtained: **Apple** = 1836, **Ball** = 2355, **Bowl** = 3884, **Cap** = 2945 and **Soda Can** = 3555. Training used 80% and 20% for testing.

- **Network Architecture (ResNet-18)**

  ResNet-18 is selected because of its proven balance between computational cost and precision. **The model contains:** 18 convolutional layers with residual blocks , 11 million parameters, pre-trained ImageNet Weights and fully-connected final layer replaced with a 5-class classifier[2].

  Model Summary:

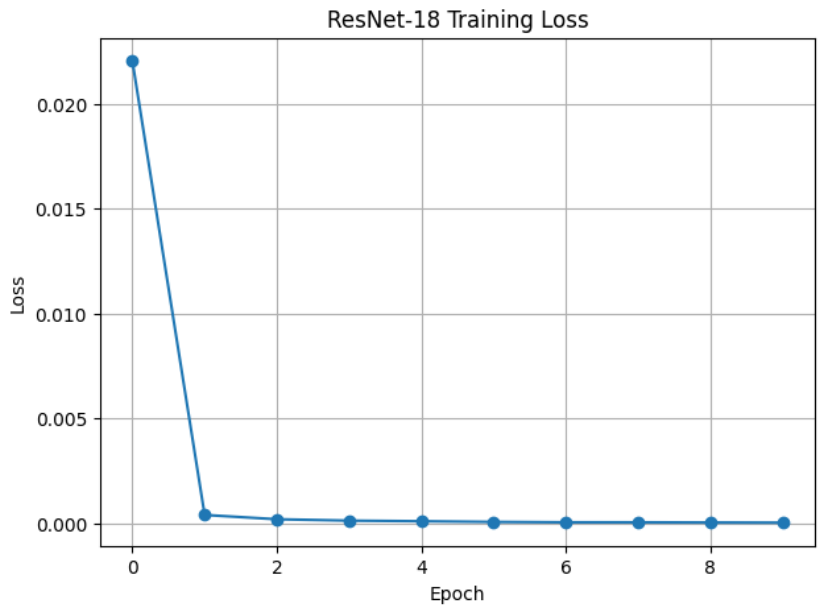
  Batch Normalization of Convolution layer stack and skipping residual connections for stable gradient[2]. Global average pooling; Linear layer: $512 \rightarrow 5$

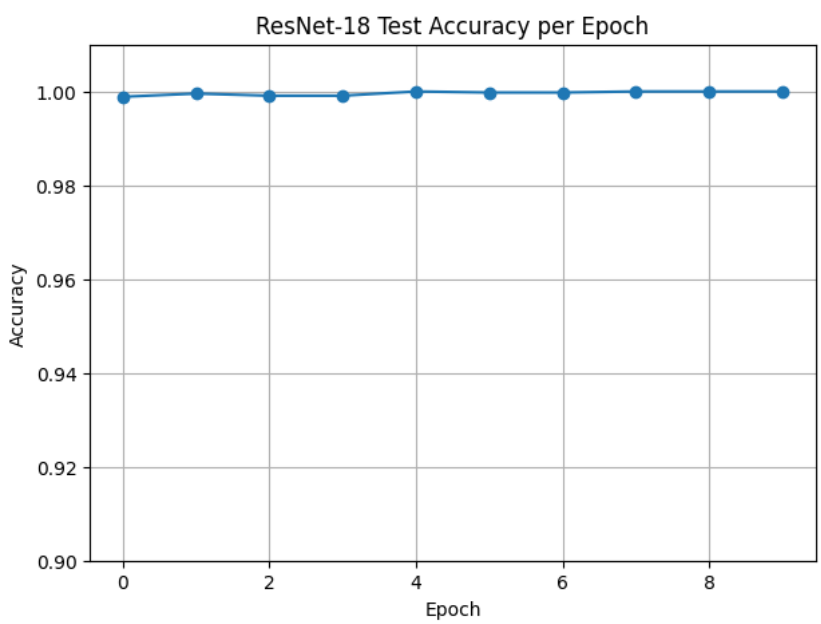
- **Hyperparameters**

  Optimizer = Adam, **Learning Rate** = 1e-4 , **Batch Size** = 32, **Epochs** = 10, **Loss Function** = CrossEntropy, **Weight Initialization** = ImageNet pretrained

- **Training Procedure**

  The model was trained in Google Colab with GPU acceleration (T4). For each epoch, we tracked the training Loss, test accuracy and training predictions for evaluation metrics.





3. CNN Results

- **Training Results**

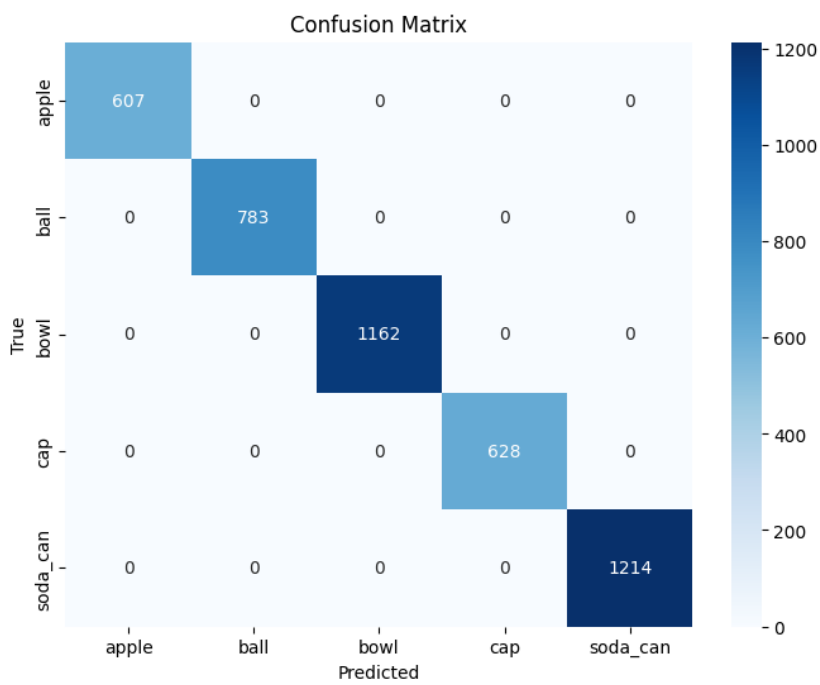
  The ResNet-18 model has achieved: Final Test Accuracy: 99.89% , Minimal overfitting due to sequence-based splitting and Stable training behavior after epoch 2.

  Training output:

```
Epoch 1/10  Loss=0.0220  Acc=0.9989
Epoch 2/10  Loss=0.0004  Acc=0.9995
Epoch 3/10  Loss=0.0002  Acc=0.9991
Epoch 4/10  Loss=0.0001  Acc=0.9991
Epoch 5/10  Loss=0.0001  Acc=1.0000
Epoch 6/10  Loss=0.0001  Acc=0.9998
Epoch 7/10  Loss=0.0000  Acc=0.9998
Epoch 8/10  Loss=0.0000  Acc=1.0000
Epoch 9/10  Loss=0.0000  Acc=1.0000
Epoch 10/10  Loss=0.0000  Acc=1.0000
```

- **Confusion Matrix Interpretation**

  All classes show near-perfect classification. Misclassifications are minimal and likely due to visual similarity between soda cans and balls in some viewpoints and variations in sequences based on illumination or pose.



- **Discussion and Future Hyperparameter Exploration**

  The perfect accuracy achieved is due to simplicity and cleanliness of dataset, use of transfer learning and absence of duplicate images. To further strengthen the results: Train deeper architectures (eg: ResNet-50, EfficientNet) and add stronger augmentation (rotation, perspective,lighting). Evaluate impact of learning rate and train without ImageNet pretraining for performance studies.

4. Object Recognition Using Local Features (ORB + BoVW + SVM)

- **ORB Feature Extraction**

  ORB (Oriented FAST and Rotated BRIEF) is chosen for final rotation invariance, low computational cost and robust binary descriptors[3]. **Parameters:** Features per image = 500, Scale factor = 1.2 and Number of levels = 8.

- **Bag-of-Visual-Words (BoVW)**

  BoVW pipeline: Extract ORB descriptors from all training images and cluster them using **MiniBatch KMeans** and Visual vocabulary size: **K=500**. Represent each image as a histogram of visual word occurrences and standardize using *StandardScaler*

- **Classification Using SVM**

  Support Vector Machine (SVM) with RBF Kernel uses **Parameters:** C=10, gamma= 'scale', and is trained on BoVW histograms. **Output:** class prediction based solely on local features.

5. ORB Results: Accuracy and Confusion Matrix

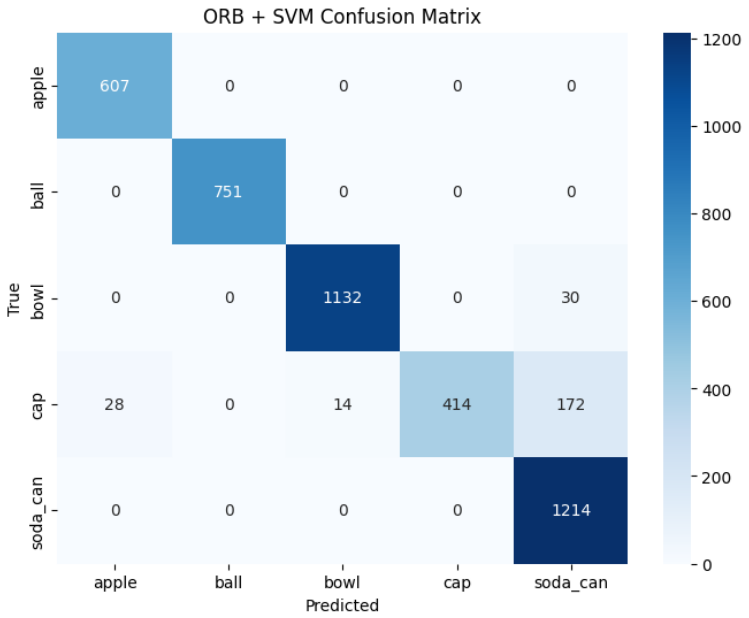
- **Overall Accuracy:** 0.9440623567171023

- **Classification Report**

```
Classification Report:

              precision    recall  f1-score   support

       apple       0.96      1.00      0.98       607
        ball       1.00      1.00      1.00       751
        bowl       0.99      0.97      0.98      1162
         cap       1.00      0.66      0.79       628
    soda_can       0.86      1.00      0.92      1214

    accuracy                           0.94      4362
   macro avg       0.96      0.93      0.94      4362
weighted avg       0.95      0.94      0.94      4362
```

- **Interpretation**

  Cap performs poorly due to weak texture which makes ORB struggle. **Soda cans** perform well because of strong edges and text. **ORB captures details locally but fails on global shape**, especially with rotation/scale variance.



6. Deep Learning vs Traditional Computer Vision: Comparison and Discussion

- <u>Accuracy Comparison</u>

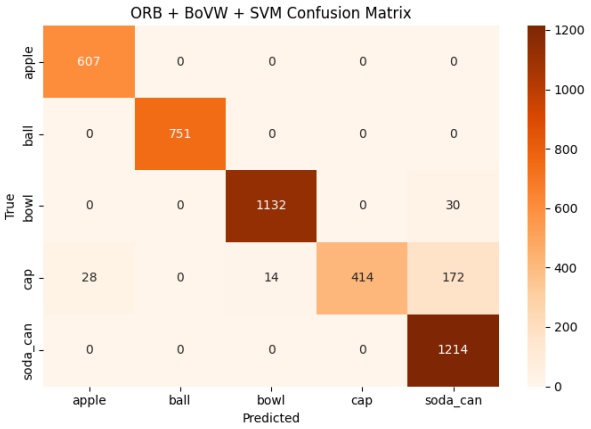
  ResNet-18 CNN $= 0.999$

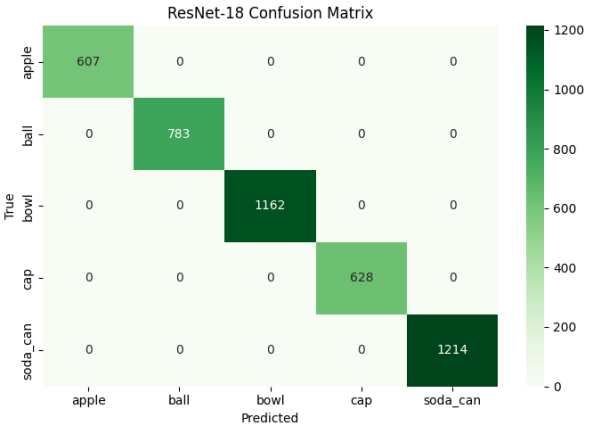
  ORB+BoVW+SVM $= 0.944$

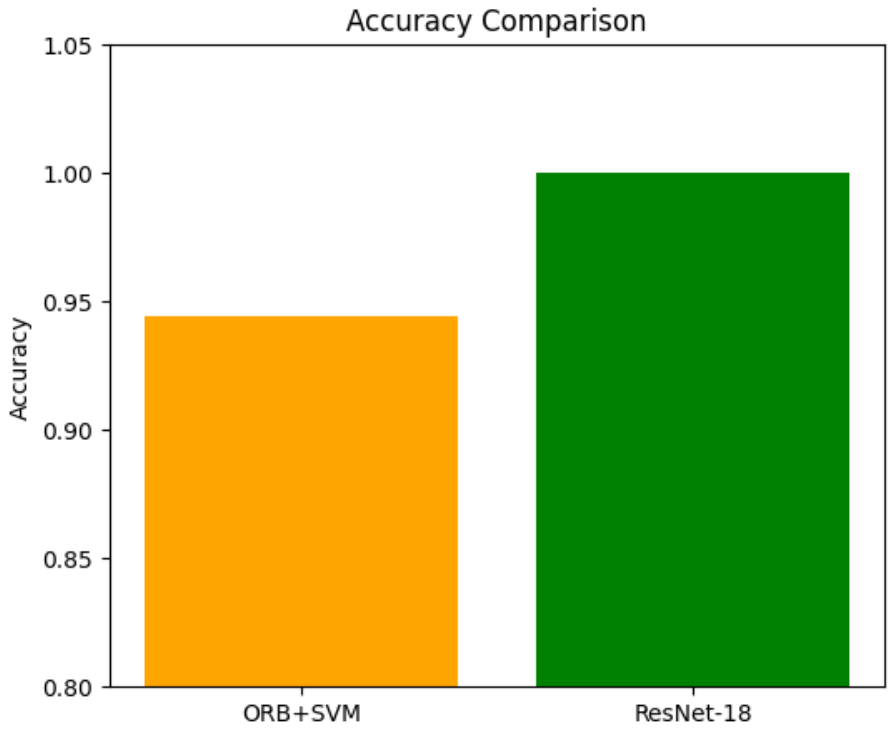
- <u>Strength and weakness</u>

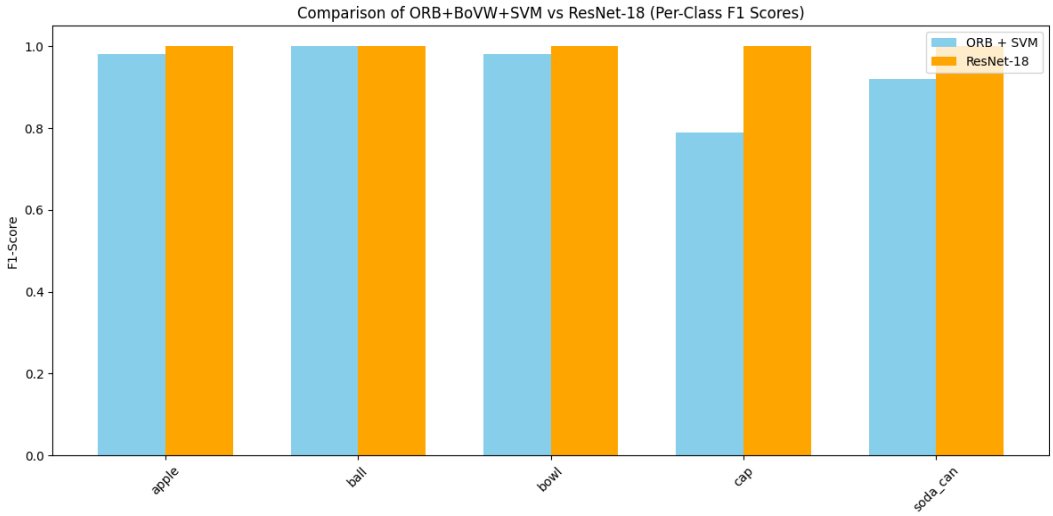
  CNN Strengths: Learns hierarchical features, robust to lighting and pose, uses global context and generalizes across sequences.

  ORB Weaknesses: Relies on hand-crafted features, sensitive to smooth textures, only captures local edges and struggles with clutter and low contrast.









7. State of the Art in Computer Vision for Robotics

- <u>**Modern Approaches**</u>

  Contemporary robotics vision uses: **CNNs** for segmentation, classification and detection[4], **Transformers** for object detection and pose estimation[5], **Self-supervised learning** for low-label environments[7]. **Multimodal RGB-D fusion networks** for grasping and environment understanding[6].

- **Deep Learning in Robotics**

  Advantages: Learns features directly from data and is robust under real-world variations, scales with dataset size and integrates seamlessly with control and planning pipelines through end-to-end learning.

  Challenges: Requires large datasets, high compute cost and difficult to interpret; May fail under domain shift.

8. Conclusion

This report implemented and compared two object recognition systems for robotic perception: **ResNet-18 CNN**, achieving 99.9% accuracy, robust and modern and **ORB + BoVW + SVM**, achieving 94% accuracy, efficient but limited .

Deep learning clearly outperforms classical methods because RGB-D images contain significant appearance variation that CNNs model more effectively. Classical methods remain educationally valuable and computationally lightweight, but are far from state-of-the-art.

9. References

- *K. Lai et al., "A Large-Scale Hierarchical Multi-View RGB-D Object Dataset," IEEE Intl. Conf. on Robotics and Automation (ICRA), 2011. [1]*
- *K. He et al., "Deep Residual Learning for Image Recognition," CVPR, 2016. [2]*
- *E. Rublee et al., "ORB: An Efficient Alternative to SIFT or SURF," ICCV, 2011. [3]*
- *D. Lowe, "Distinctive Image Features from Scale-Invariant Keypoints," IJCV, 2004. [4]*
- *A. Dosovitskiy et al., "An Image is Worth 16x16 Words: Vision Transformer," ICLR, 2021. [5]*
- *A. Depierre et al., "Deep Learning for Grasping in Robotics: A Survey," IEEE Robotics Automation Letters, 2021. [6]*
- *OpenAI et al., "RT-2: Vision-Language-Action Models for Robotic Control," 2023 [7]*

10. Appendix

```
from google.colab import drive
drive.mount('/content/drive')





#-----ResNet-18 CNN---------#


# Imports
import os
import cv2
import glob
import numpy as np
from PIL import Image, ImageFile

import torch
import torch.nn as nn
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, confusion_matrix,
↪  classification_report

import matplotlib.pyplot as plt
import seaborn as sns

# Fix truncated PNG errors
ImageFile.LOAD_TRUNCATED_IMAGES = True

# GPU check
device = "cuda" if torch.cuda.is_available() else "cpu"
print("Using device:", device)




#Remove Duplicate Images

import os, hashlib
ROOT = "/content/drive/MyDrive/RGBD/rgbd-dataset"
```

```python
def md5(path, blocksize=2**20):
    h = hashlib.md5()
    with open(path,'rb') as f:
        while True:
            b = f.read(blocksize)
            if not b: break
            h.update(b)
    return h.hexdigest()

hash_to_paths = {}
for cls in os.listdir(ROOT):
    class_dir = os.path.join(ROOT, cls)
    if not os.path.isdir(class_dir): continue
    for root, dirs, files in os.walk(class_dir):
        for f in files:
            if f.lower().endswith(".png"):
                full = os.path.join(root,f)
                h = md5(full)
                hash_to_paths.setdefault(h, []).append(full)

dups = {h:ps for h,ps in hash_to_paths.items() if len(ps)>1}
print("Number of duplicate-image groups:", len(dups))
# print a few groups
for i,(h,ps) in enumerate(dups.items()):
    if i>5: break
    print("=== group", i, "hash", h)
    for p in ps: print(" ", p)



import os, glob, hashlib
from sklearn.model_selection import train_test_split

ROOT = "/content/drive/MyDrive/RGBD/rgbd-dataset"
CLASSES = ["apple", "ball", "bowl", "cap", "soda_can"]

def md5(path, blocksize=2**20):
    h = hashlib.md5()
    with open(path,'rb') as f:
        while True:
            b = f.read(blocksize)
            if not b: break
            h.update(b)
    return h.hexdigest()

# 1) Load all sequences + images
data = {c:{} for c in CLASSES}
for c in CLASSES:
```

```
    cdir = os.path.join(ROOT, c)
    if not os.path.isdir(cdir): continue

    for seq in sorted(os.listdir(cdir)):
        sp = os.path.join(cdir, seq)
        if not os.path.isdir(sp): continue
        imgs = glob.glob(sp + "/*.png")
        imgs = [p for p in imgs if "_mask" not in p and "_depth" not
        ↪  in p]
        if imgs:
            data[c][seq] = sorted(imgs)

# 2) Remove duplicates INSIDE EACH sequence (but keep one)
removed = 0
for c in CLASSES:
    for seq, imgs in data[c].items():
        seen = {}
        unique = []
        for img in imgs:
            h = md5(img)
            if h not in seen:
                seen[h] = img
                unique.append(img)
            else:
                removed += 1
        data[c][seq] = unique

print("Removed duplicates inside sequences:", removed)

# 3) Sequence-level train/test split
train_paths, train_labels = [], []
test_paths, test_labels = [], []

for c in CLASSES:
    seqs = list(data[c].keys())
    if len(seqs) == 0:
        continue

    if len(seqs) < 3:
        train_seq = seqs[:-1]
        test_seq = seqs[-1:]
    else:
        train_seq, test_seq = train_test_split(seqs, test_size=0.2,
        ↪  random_state=42)

    for s in train_seq:
        train_paths += data[c][s]
        train_labels += [c] * len(data[c][s])
```

```
    for s in test_seq:
        test_paths += data[c][s]
        test_labels += [c] * len(data[c][s])

print("Train size:", len(train_paths))
print("Test size:", len(test_paths))

# 4) Final leakage check
train_hashes = {md5(p) for p in train_paths}
test_hashes = {md5(p) for p in test_paths}

print("Cross-split duplicates:", len(train_hashes & test_hashes))



#Training

from sklearn.metrics import accuracy_score

EPOCHS = 10

train_losses = []
test_accs = []

for epoch in range(EPOCHS):
    model.train()
    running_loss = 0

    for imgs, labels in train_loader:
        imgs, labels = imgs.to(device), labels.to(device)

        optimizer.zero_grad()
        outputs = model(imgs)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()

    train_losses.append(running_loss / len(train_loader))

    # ---- Evaluation ----
    model.eval()
    all_preds, all_true = [], []

    with torch.no_grad():
        for imgs, labels in test_loader:
```

```
            imgs, labels = imgs.to(device), labels.to(device)
            out = model(imgs)
            preds = torch.argmax(out, dim=1)

            all_preds.extend(preds.cpu().numpy())
            all_true.extend(labels.cpu().numpy())

    acc = accuracy_score(all_true, all_preds)
    test_accs.append(acc)

    print(f"Epoch {epoch+1}/{EPOCHS}  Loss={train_losses[-1]:.4f}
    ↪  Acc={acc:.4f}")


print("\nFINAL ACCURACY:", accuracy_score(all_true, all_preds))
print("\nClassification Report:")
print(classification_report(all_true, all_preds,
↪  target_names=CLASSES))
cm = confusion_matrix(all_true, all_preds)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=CLASSES, yticklabels=CLASSES)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()









#-----ORB + Bag of Visual Words (BoVW) + SVM-------#




import cv2
import numpy as np
from sklearn.cluster import MiniBatchKMeans
from sklearn.svm import SVC
```

```python
from sklearn.metrics import accuracy_score, classification_report,
↪  confusion_matrix
import matplotlib.pyplot as plt
import seaborn as sns


# 1. ORB Feature Extraction Function

def extract_orb_descriptors(image_paths, max_kp=500):
    orb = cv2.ORB_create(nfeatures=max_kp)
    all_desc = []

    print("Extracting ORB descriptors...")

    for p in image_paths:
        img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
        if img is None:
            continue

        kp, des = orb.detectAndCompute(img, None)
        if des is not None:
            all_desc.append(des)

    return all_desc





# Build Vocabulary (K-Means on descriptors)

def build_vocabulary(des_list, vocab_size=500):
    print("Stacking descriptors...")
    stacked = np.vstack(des_list)   # shape [N_descriptors, 32]

    print("Clustering descriptors (MiniBatchKMeans)...")
    kmeans = MiniBatchKMeans(n_clusters=vocab_size,
                             batch_size=2000,
                             max_iter=100,
                             verbose=1)
    kmeans.fit(stacked)

    return kmeans
```

```python
# Convert images → histogram features

def compute_bow_histograms(kmeans, image_paths, max_kp=500):
    orb = cv2.ORB_create(nfeatures=max_kp)
    vocab_size = kmeans.n_clusters

    histograms = []

    print("Computing histograms for", len(image_paths), "images...")

    for p in image_paths:
        img = cv2.imread(p, cv2.IMREAD_GRAYSCALE)
        if img is None:
            histograms.append(np.zeros(vocab_size))
            continue

        kp, des = orb.detectAndCompute(img, None)

        if des is None:
            histograms.append(np.zeros(vocab_size))
            continue

        words = kmeans.predict(des)
        hist, _ = np.histogram(words, bins=np.arange(vocab_size+1))
        histograms.append(hist)

    return np.array(histograms)




#ORB + SVM Pipeline
# ----------------------------------------

# Extract ORB descriptors from training images only
train_des = extract_orb_descriptors(train_paths)

# Build vocabulary
VOCAB_SIZE = 600
kmeans = build_vocabulary(train_des, vocab_size=VOCAB_SIZE)

# Compute BoW histograms
X_train = compute_bow_histograms(kmeans, train_paths)
X_test  = compute_bow_histograms(kmeans, test_paths)

# Train SVM classifier
print("Training SVM...")
```

```
svm = SVC(kernel="linear", probability=True)
svm.fit(X_train, train_labels)

print("SVM trained.")




# Evaluate
pred = svm.predict(X_test)

print("\nAccuracy:", accuracy_score(test_labels, pred))
print("\nClassification Report:\n")
print(classification_report(test_labels, pred))
cm = confusion_matrix(test_labels, pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt="d", cmap="Blues",
            xticklabels=CLASSES, yticklabels=CLASSES)
plt.title("ORB + SVM Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("True")
plt.show()


#-----------Comparison plots----------------------#


#Accuracy Comparison
import matplotlib.pyplot as plt

orb_acc = accuracy_score(test_labels, pred)
cnn_acc = accuracy_score(all_true, all_preds)

plt.figure(figsize=(6,5))
plt.bar(["ORB+SVM", "ResNet-18"], [orb_acc, cnn_acc], color=["orange",
↪  "green"])
plt.ylim(0.8, 1.05)
plt.title("Accuracy Comparison")
```

```
plt.ylabel("Accuracy")
plt.show()


#Confusion Matrix Comaprison
from sklearn.metrics import confusion_matrix
import seaborn as sns
import numpy as np

fig, axes = plt.subplots(1, 2, figsize=(14,5))

# ORB confusion matrix
cm_orb = confusion_matrix(test_labels, pred)
sns.heatmap(cm_orb, annot=True, fmt="d", cmap="Oranges",
            xticklabels=CLASSES, yticklabels=CLASSES, ax=axes[0])
axes[0].set_title("ORB + BoVW + SVM Confusion Matrix")
axes[0].set_xlabel("Predicted")
axes[0].set_ylabel("True")

# CNN confusion matrix
cm_cnn = confusion_matrix(all_true, all_preds)
sns.heatmap(cm_cnn, annot=True, fmt="d", cmap="Greens",
            xticklabels=CLASSES, yticklabels=CLASSES, ax=axes[1])
axes[1].set_title("ResNet-18 Confusion Matrix")
axes[1].set_xlabel("Predicted")
axes[1].set_ylabel("True")

plt.tight_layout()
plt.show()


#CNN Training Curve

# Training Loss
plt.figure(figsize=(7,5))
plt.plot(train_losses, marker="o")
plt.title("ResNet-18 Training Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.grid()
plt.show()


# Test Accuracy
plt.figure(figsize=(7,5))
plt.plot(test_accs, marker="o")
plt.title("ResNet-18 Test Accuracy per Epoch")
plt.xlabel("Epoch")
```

```
plt.ylabel("Accuracy")
plt.ylim(0.9, 1.01)
plt.grid()
plt.show()




#Per class F1 Score Comparison

CLASSES = ["apple", "ball", "bowl", "cap", "soda_can"]

test_labels = {
    "apple": 0.98,
    "ball": 1.00,
    "bowl": 0.98,
    "cap": 0.79,
    "soda_can": 0.92
}
all_true = {
    "apple": 1.00,
    "ball": 1.00,
    "bowl": 1.00,
    "cap": 1.00,
    "soda_can": 1.00
}

pred = [test_labels[c] for c in CLASSES]
all_preds = [all_true[c] for c in CLASSES]


# ---- Plot ----
x = np.arange(len(CLASSES))
width = 0.35

plt.figure(figsize=(12, 6))
plt.bar(x - width/2, pred, width, label="ORB + SVM", color="skyblue")
plt.bar(x + width/2, all_preds, width, label="ResNet-18",
↪  color="orange")

plt.xticks(x, CLASSES, rotation=45)
plt.ylabel("F1-Score")
plt.title("Comparison of ORB+BoVW+SVM vs ResNet-18 (Per-Class F1
↪  Scores)")
plt.legend()
plt.tight_layout()
plt.show()
```